**Dept. of Computer Science & Software Eng., Concordia University**
**COMP 6481 --- Fall 2026**

# Programming and Problem Solving

**Assignment 2 --- Due Sunday, November 01, 2026**

---

**Purpose:** The purpose of this assignment is to allow you practice Array Lists and Linked Lists, as well as other previous object-oriented concepts.

---

"Success is the sum of small efforts, repeated day in and day out."

--- Robert Collier

Space exploration requires extraordinary levels of planning, coordination, and precision. Throughout a space mission, all the mission-control teams must continuously manage hundreds of activities involving spacecraft systems, scientific experiments, navigation, communications, crew operations, and safety procedures.

Imagine a system called **Artemis Mission Control** — a Java-based mission-management simulator designed to organize and monitor the activities of a lunar mission. As a software developer working with the Mission Control software team, your responsibility is to develop a system that helps flight controllers organize mission tasks, identify critical operations, monitor scheduled activities, record completed operations, and maintain a history of the mission.

Throughout a lunar mission, different operations must be performed at specific times. Some operations, such as routine scientific observations, may have moderate priority, while others — such as navigation corrections, communication recovery, or life-support checks — may be mission-critical.

In this assignment, you will design a **Java-based Artemis Lunar Mission Control Simulator** that allows mission controllers to create, modify, prioritize, complete, clone, search, and monitor mission operations. You will implement the system using **object-oriented programming**, **custom exceptions**, **ArrayList**, **LinkedList**, object cloning, deep copying, and comparison techniques.

Your program will maintain mission operations using two primary collections:

- An **ArrayList** containing all current and upcoming mission tasks.
- A **LinkedList** containing completed mission tasks, with the most recently completed task appearing first.

The system will also include a custom linked-list implementation representing a simplified **Mission Timeline**.

## Assignment Overview

You will create a Java program that:

1. Manages mission operations using **ArrayList** and **LinkedList** structures.
2. Supports adding, updating, deleting, searching, cloning, & completing mission tasks.
3. Identifies mission-critical operations based on priority.
4. Detects mission tasks whose scheduled execution time has passed.

5. Handles invalid, incomplete, or duplicate information using **custom exceptions**.
6. Provides user interaction through a **console-based menu**.
7. Implements object **cloning**, copy constructors, deep copies, and object comparison.
8. Generates summaries of important and completed mission operations.
9. Demonstrates appropriate use of **encapsulation, interfaces, classes, objects, and collection** data structures.

## Output Possibilities

Depending on the user's commands, your program may print messages such as:

a) "Mission task [NAV-101] successfully added to the active mission list."
b) "Mission task [COM-205] completed and moved to the mission history."
c) "Mission task [SCI-110] already exists. Duplicate entries are not allowed."
d) "Mission task [SYS-302] removed from the mission plan."
e) "Error: Mission task information is missing or invalid."
f) "Warning: 3 mission tasks have passed their scheduled execution time."
g) "CRITICAL: Life Support System Check requires immediate attention."
h) "The top 3 mission tasks based on priority are displayed below."
i) "Mission task [NAV-101] found after 4 iterations."
j) "Mission summary successfully generated."

## Detailed Task Description

---

### I. Create an Interface

Create an interface named: MissionCritical

The interface must contain the following method:

***public boolean isCritical()***;

The method returns:

- true if the supplied mission task has a priority between **8 and 10**, inclusive.
- false otherwise.

Priority will use the following scale:

| Priority | Meaning |
|---|---|
| **1–3** | Low priority |
| **4–7** | Normal/Important |
| **8–10** | Mission Critical |

Examples of mission-critical operations could include:

- Life-support system checks
- Navigation corrections
- Communication recovery
- Spacecraft system failures
- Emergency procedures

## II. MissionTask Class

Implement a class **MissionTask** that **implements the MissionCritical interface**.

### Attributes

- String taskID; A unique identifier for the operation. (e.g. NAV-101, COM-201, etc.)
- String taskName; The descriptive name of the mission operation. (*e.g.* Life Support Diagnostics, Lunar Navigation Correction, Photograph Lunar Surface, *etc.*)
- String category; The type of mission activity. (e.g. NAVIGATION, SCIENCE, etc.)
- int priority; An integer from 1 to 10 where 10 being the highest priority.
- String scheduledDate; Operation's scheduled date in YYYY-MM-DD format.
- int estimatedDuration; Estimated minutes required to complete the operation.
- boolean isCompleted; Indicates whether the mission operation has been completed.
- ArrayList<String> requiredSystems; An ArrayList identifying spacecraft systems involved in the operation. (*e.g.* Navigation Computer, Orion Guidance System, *etc.*)

### Requirements

#### a) Parameterized Constructor:

Create a parameterized constructor that initializes all attributes using values supplied by the user. For instance,

```
MissionTask task = new MissionTask("NAV-101", "Lunar Navigation Correction",
                                   "NAVIGATION", 9, "2027-05-15", 45, false,
                                   systems);
```

The constructor ***must ensure that the supplied information is valid***.

#### b) Copy Constructor:

Takes a MissionTask object and a new taskID, creating a copy of the original task but with the unique task ID. For instance,

***MissionTask copy = new MissionTask(originalTask, "NAV-102");***

The new object's **requiredSystems** list must be a **deep copy**. Changing the copied list must therefore NOT change the list belonging to the original object.

#### c) clone() Method:

Implement **clone()** that creates and returns a **deep copy** of the current MissionTask.

***MissionTask copiedTask = originalTask.clone();***

The cloned object must have its own independent copy of ***requiredSystems***. The program should request a new unique task ID before adding cloned task to the active mission list.

#### d) toString() and equals() Methods:

- toString() overrides the toString method and displays all mission information clearly.
- equals() overrides the equals method from the Object class and compares two mission task objects. Two mission tasks are considered equal if all their mission information is identical except taskID.

e) **isCritical():**

Implements the method inherited from MissionCritical. It must return true if the task's priority is between 8 and 10, inclusive.

f) **Accessors and Mutators:**

Provide appropriate getter and setter methods for all attributes. All attributes should be declared private to demonstrate proper encapsulation.

---

## III. MissionTimeline Class

Create a class named: ***MissionTimeline***. This class represents a ***custom linked list*** containing mission tasks. **Do not use Java's built-in LinkedList** to implement this class.

### (a) Inner Class: MissionTaskNode

Inside MissionTimeline, create an inner class: ***MissionTaskNode***.

Each node must contain: ***MissionTask*** task and ***MissionTaskNode*** next;

Provide the following:

- Default constructor
- Parameterized constructor
- Copy constructor
- clone() method
- Accessor and Mutator methods

### (b) MissionTimeline Attributes

The MissionTimeline class must contain:

- MissionTaskNode head
- int size

**head** represents the first mission operation in the timeline. **size** represents the number of operations currently stored.

### (c) MissionTimeline Methods

Implement the following methods.

1. addToStart(MissionTask task)
   – Adds a mission task to the beginning of the timeline.
2. insertAtIndex(MissionTask task, int index)
   – Inserts a mission task at the specified index. If the supplied index is invalid, throw NoSuchElementException.
3. deleteFromIndex(int index)
   – Deletes the mission task at the specified index, throws NoSuchElementException if the index is invalid.
4. deleteFromStart()
   – Removes the first mission task from the timeline. The head reference and size must be updated correctly.

5. replaceAtIndex(MissionTask task, int index)
   – Replaces the mission task stored at the specified index.
6. find(String taskID)
   – Searches for a mission task using its unique ID. If the task cannot be found, an appropriate message should be displayed.
   The method must:
   - ✓ Traverse the linked list.
   - ✓ Count the number of nodes examined.
   - ✓ Print the number of iterations required.
   - ✓ Return a **deep copy** of the matching task.

   Example: Mission task COM-103 found after 4 iterations.
7. contains(String taskID)
   – Returns true if the mission task exists in the timeline.
8. equals(MissionTimeline other)
   – Two mission timelines are considered equal when they contain identical mission tasks in the same order. The comparison must use the equals() from MissionTask.
9. getSize()
   – Returns the number of mission tasks contained in the timeline.
10. displayTimeline ()
    – Displays all mission tasks in their current timeline order.

---

## IV. MissionControlManager Class

Create a class named: MissionControlManager. This class manages all the active as well as completed mission operations.

**Attributes:**

- ArrayList<MissionTask> activeTasks
  - Contains all mission tasks that are currently planned or in progress.
- LinkedList<MissionTask> completedTasks
  - Contains all completed mission operations. Completed tasks must be inserted at the beginning of the list such that most recently completed operation appears first.

**Methods:**

1. addMissionTask(MissionTask task)
   – Adds mission task to activeTasks. Before inserting the task, check whether it already exists. If a duplicate is detected, throw: DuplicateMissionTaskException.
2. updateMissionTask(String taskID)
   – Allows mission controllers to modify information such as: Task name, Priority, Scheduled date, Estimated duration, Category, Required systems, *etc.* The task ID must remain unique.
3. completeMissionTask (String taskID)
   – Find the mission task inside activeTasks, then mark it as completed. Remove it from activeTasks. Insert it at the beginning of completedTasks.
   For example: completedTasks.addFirst(task);
   **Output:** Mission task NAV-101 completed and moved to mission history.

4. deleteMissionTask(String taskID)
   – Search activeTasks and completedTasks. Remove the matching mission task.
5. showCriticalTasks()
   – Display all active tasks whose priority is between 8 and 10. The program must use isCritical() when determining whether an operation is critical.
6. showTopMissionTasks()
   – Display **three active mission tasks with the highest priorities**. You may create a temporary copy of the ArrayList when sorting so that the original mission list is not unnecessarily rearranged.
7. showOverdueMissionTasks ()
   – Compare each active mission task's scheduledDate with the system's current date. Use Java's date API, such as LocalDate. A mission task is overdue when:
   ***scheduledDate < currentDate and isCompleted == false.***
8. cloneMissionTask (String taskID)
   – Find the specified mission task. Create a deep copy. Request a new unique mission task ID. Add the cloned task to activeTasks. The original & cloned objects must not share the same ***requiredSystems*** ArrayList.
9. findMissionTask(String taskID)
   – Search for the mission operation in the active and completed mission collections. Return the appropriate mission task if found.
10. generateMissionSummary()
    – Generate a summary containing information such as:

    ========= ARTEMIS MISSION SUMMARY =========

    Active Mission Tasks: 12
    Completed Mission Tasks: 8
    Critical Active Tasks: 3
    Overdue Tasks: 1

    Average Estimated Duration:
    42.5 minutes

    Highest Priority Operation:
    LIFE-210 - Oxygen System Diagnostic

    Most Recently Completed Operation:
    COM-105 - Communication Array Test

    ==========

---

## V. Custom Exceptions

Implement the following two custom exception classes.

1. **InvalidMissionTaskException**

   This exception must be thrown when mission-task information is invalid. Examples include: Missing task ID, Missing task name, Invalid priority, Invalid scheduled date, Negative or zero estimated duration, Missing category.

   - Default error message: "Error: Invalid or missing mission task information."

2. **DuplicateMissionTaskException**

   This exception must be thrown when an attempt is made to insert a duplicate task.

   - Default message: "Error: Mission task already exists in the mission plan."

---

### VI. ArtemisMissionControlMain Class

Your main driver class will provide the menu interface and program logic.

**Steps:**

(A) **Create** an instance of MissionControlManager.

(B) **Display menu** with options:

Your program must repeatedly display a menu similar to:

```
========================================
      ARTEMIS MISSION CONTROL
========================================
1. Add Mission Task
2. Update Mission Task
3. Complete Mission Task
4. Delete Mission Task
5. View Active Mission Tasks
6. View Completed Mission Tasks
7. View Critical Mission Tasks
8. View Top 3 Mission Tasks
9. View Overdue Mission Tasks
10. Clone Mission Task
11. Search Mission Task
12. Generate Mission Summary
13. Test Mission Timeline
14. Exit

Enter your selection:
```

(C) **Process user input** via Scanner.

Use: Scanner to collect information from the user. The program must validate user input before performing an operation.

(D) **Handle exceptions** gracefully.

Use the appropriate: try and catch blocks to handle: InvalidMissionTaskException, DuplicateMissionTaskException, NoSuchElementException, NumberFormatException. The application must not terminate unexpectedly because the user entered invalid information. Instead, display a meaningful message and return user to the menu.

(E) **Test required OOP features**

You must explicitly demonstrate: 1. Parameterized constructor 2. Copy constructor, 3. Deep copy, 4. clone(), 5. equals(), 6. Interface (*i.e.* demonstrate the isCritical() method for both a normal-priority and mission-critical task, 7. ArrayList (*i.e.* demonstrate insertion, deletion, searching, updating, and traversal of active mission tasks.), 8. LinkedList (*i.e.* demonstrate that completed mission operations are maintained with the most recently completed operation appearing first.), 9. Custom Linked List (*i.e.* demonstrate following methods from MissionTimeline: addToStart(), insertAtIndex(), deleteFromIndex(), replaceAtIndex(), find(), contains(), equals()).

### VII Sample Mission Data

You may use the following operations when testing your program.

| Task ID | Mission Operation | Category | Priority | Duration |
|---|---|---|---|---|
| NAV-101 | Lunar Navigation Correction | NAVIGATION | 9 | 45 min |
| COM-102 | Deep Space Communication | COMMUNICATION | 8 | 30 min |
| SCI-103 | Lunar Surface Photography | SCIENCE | 5 | 60 min |
| LIFE-104 | Oxygen System Diagnostic | LIFE_SUPPORT | 10 | 25 min |
| ENG-105 | Solar Array Inspection | ENGINEERING | 7 | 40 min |
| SCI-106 | Radiation Measurement | SCIENCE | 6 | 90 min |
| CREW-107 | Crew Health Assessment | CREW | 7 | 20 min |

---

### VIII Sample Run

```
========================================
       ARTEMIS MISSION CONTROL
========================================

1. Add Mission Task
2. Update Mission Task
3. Complete Mission Task
4. Delete Mission Task
5. View Active Mission Tasks
6. View Completed Mission Tasks
7. View Critical Mission Tasks
8. View Top 3 Mission Tasks
9. View Overdue Mission Tasks
10. Clone Mission Task
11. Search Mission Task
12. Generate Mission Summary
13. Test Mission Timeline
14. Exit

Enter your selection: 1

Enter Mission Task ID:
NAV-101

Enter Task Name:
Lunar Navigation Correction

Enter Category:
NAVIGATION

Enter Priority (1-10):
9
```

```
Enter Scheduled Date:
2027-05-15

Enter Estimated Duration:
45

Enter number of required systems:
2

System 1:
Navigation Computer

System 2:
Orion Guidance System

Mission task NAV-101 successfully added to the active mission list.

========================================
       ARTEMIS MISSION CONTROL
========================================

1. Add Mission Task
2. Update Mission Task
3. Complete Mission Task
4. Delete Mission Task
5. View Active Mission Tasks
6. View Completed Mission Tasks
7. View Critical Mission Tasks
8. View Top 3 Mission Tasks
9. View Overdue Mission Tasks
10. Clone Mission Task
11. Search Mission Task
12. Generate Mission Summary
13. Test Mission Timeline
14. Exit

Enter your selection: 7

========= CRITICAL MISSION OPERATIONS =========

LIFE-104
Oxygen System Diagnostic
Priority: 10

NAV-101
Lunar Navigation Correction
Priority: 9

COM-102
Deep Space Communication Check
Priority: 8

Total Critical Operations: 3
```

```
========================================
       ARTEMIS MISSION CONTROL
========================================

1. Add Mission Task
2. Update Mission Task
3. Complete Mission Task
4. Delete Mission Task
5. View Active Mission Tasks
6. View Completed Mission Tasks
7. View Critical Mission Tasks
8. View Top 3 Mission Tasks
9. View Overdue Mission Tasks
10. Clone Mission Task
11. Search Mission Task
12. Generate Mission Summary
13. Test Mission Timeline
14. Exit

Enter your selection: 3

Enter Mission Task ID:
NAV-101

Mission task NAV-101 completed successfully.
Task moved to mission history.

Most Recent Completed Task:
NAV-101 - Lunar Navigation Correction
```

---

**IX. Programming Requirements** Your solution must:

1. Use appropriate object-oriented programming principles.
2. Declare object attributes as private.
3. Use constructors appropriately.
4. Override toString().
5. Override equals().
6. Implement a copy constructor.
7. Implement object cloning.
8. Demonstrate a genuine deep copy.
9. Implement and use an interface.
10. Use ArrayList<MissionTask>.
11. Use LinkedList<MissionTask>.
12. Implement the custom MissionTimeline linked structure.
13. Implement at least two custom exception classes.
14. Use Java exception handling.
15. Validate user input.
16. Use Java's date functionality for date comparisons.
17. Provide a functional console-based interface.
18. Avoid unnecessary global/static variables.
19. Use meaningful variable and method names.
20. Include appropriate comments and documentation.

## X. Suggested Java Files

Your completed project should contain at least the following files:

- MissionCritical.java
- MissionTask.java
- MissionTimeline.java
- MissionControlManager.java
- InvalidMissionTaskException.java
- DuplicateMissionTaskException.java
- ArtemisMissionControlMain.java

---

## XI. Learning Objectives

Upon completing this assignment, students should be able to:

- Design a multi-class Java application using OOP principles.
- Apply encapsulation to protect object data.
- Implement Java interfaces.
- Understand and implement object equality.
- Create copy constructors.
- Explain the difference between shallow and deep copying.
- Clone objects correctly.
- Store and manipulate objects using ArrayList.
- Store and manipulate objects using LinkedList.
- Understand how linked-list nodes and references operate.
- Traverse and search a linked data structure.
- Throw and handle custom exceptions.
- Validate user input.
- Work with Java's date API.
- Design a menu-driven Java application.
- Select appropriate data structures for different programming requirements

---

**Some general information:**

a. Do not use any external libraries or existing software to produce what is needed; that will directly result in a 0 mark!

b. Again, your program must work for any input data. The sample input provided with this assignment is only a possible version, and must not be considered as the general case when writing your code.

**General Guidelines When Writing Programs:**

- Include the following comments at the top of your source codes.
  // Assignment (include number)
  // Question: (include question/part number, if applicable)
  // Written by: (include your name and student id)
  // ----------
- In a comment, provide a general explanation of what your program does. As the programming questions get more complex, the explanations will get lengthier.
- Include comments in your program describing the main steps in your program.

- Display a welcome message which includes your name(s).
- Display clear prompts for users when you are expecting the user to enter data from the keyboard.
- All output must be displayed with clear messages and in an easy-to-read format.
- End your program with a closing message so that the user knows that the program has terminated.

## SUBMISSION INSTRUCTIONS

**Submission format:** All assignment-related submissions must be adequately archived in a ZIP file using your ID(s) and last name(s) as file name. The submission itself must contain your name(s) and student ID(s). Use "official" name only – no abbreviations/nick names; capitalize the "last" name. Inappropriate submissions will be heavily penalized. **IMPORTANT:** For this assignment, a demo for about 15 minutes will take place with the marker. You **must** attend the demo and be able to explain their program to the marker. The demo schedule will be determined and announced by the markers, and students must reserve a time slot for the demo.

## Now, please read very carefully:

- **If you fail to demo, a zero mark is assigned regardless of your submission.**
- **If you book a demo time, and do not show up, for whatever reason, you will be allowed to reschedule a second demo but a penalty of 50% will be applied.**
- **Failing to demo at second appointment will result in zero marks and no more chances will be given under any conditions.**

## EVALUATION CRITERIA

| Category | Criteria | Points |
|---|---|---|
| Documentation | JavaDoc documentation for classes & important methods; meaningful comments where needed | **1.0** |
| Interfaces and OOP Classes | Correct implementation of required interface and classes; encapsulation, constructors, toString(), equals(), copy constructor, cloning and deep copy | **2.0** |
| Exception Handling and Validation | Correct implementation and use of custom exceptions; invalid input is handled gracefully without crashing the program | **1.0** |
| ArrayList and LinkedList Operations | Correct use of ArrayList for active mission tasks and LinkedList for completed tasks; adding, deleting, updating, searching, completing, and moving tasks between collections work correctly | **2.0** |
| Custom MissionTimeline Linked List | Correct implementation of nodes and required operations including insert, delete, replace, search, contains(), and comparison | **2.0** |
| Testing and Output Accuracy | Menu works correctly; required features are demonstrated; output is accurate and clearly formatted; program handles normal & edge cases | **2.0** |
| **Total** | | **10.0** |